Implementation of TipFlow for TipJar+

1. Smart Contracts

We will develop three Solidity smart contracts – TipProxy, WithdrawProxy, and DepositProxy – to handle on-chain tipping flows with the specified fees. Each contract will include robust security measures, configurability, and thorough unit tests. Key details for each:

TipProxy (3.5% fee from fan): This contract acts as an intermediary when a fan sends a tip in USDC (or another stablecoin) on-chain. The contract will split the tip, automatically routing 3.5% to the platform's fee address and 96.5% to the creator's address. We'll implement this by using a basis points mechanism (e.g. feeBps = 350 for 3.5%) to calculate the fee on each transfer, avoiding floating point issues. For example, if a fan sends 100 USDC, the contract will take 3.5 USDC as fee and forward 96.5 USDC to the creator. The contract will support ERC-20 token transfers via transferFrom: the fan will approve TipProxy to spend the tip amount, then call sendTip(creator, amount) on TipProxy. The function will calculate the fee and use transferFrom to move the fee to the fee recipient and the remainder to the creator's address. Using this pattern means the contract itself holds no balance except transiently during execution, reducing risk. We will emit an event (e.g. TipSent(fan, creator, amount, fee)) for off-chain monitoring. Security measures include reentrancy guards (to prevent any callback from an ERC-20 token causing reentrance) and input validation (e.g. require a nonzero amount and valid addresses). The contract will have an owner (the TipJar+ platform) with the ability to update the fee percentage or fee recipient address if needed, but ordinary users cannot modify these. This configurability ensures we can adjust fees or upgrade the fee receiver without redeploying the entire system. Unit tests will cover fee calculation (ensuring 3.5% is correctly taken even on edge cases and rounding), event emission, and permission controls on updating config.

WithdrawProxy (3.5% fee on creator withdrawal): This contract facilitates creators withdrawing their earnings and applies a 3.5% fee to the creator upon withdrawal. If the platform custodial model holds funds on-chain for creators, the creator would call this contract to retrieve their balance. In our architecture, however, creators' funds are primarily held in their Circle custodial wallet or forwarded directly to them. Thus, the WithdrawProxy is used if/when funds are stored in a smart contract escrow on behalf of the creator. For example, if tips are pooled in a contract (rather than sent directly), a creator can call withdraw(amount) which triggers a transfer of the requested amount minus 3.5% to the creator's own wallet. The 3.5% fee is sent to the platform treasury. We will implement similar math (using basis points) to calculate the fee portion. This contract will also include safety checks like pausing (using OpenZeppelin's Pausable) and reentrancy guard on the withdraw function. Only the creator (or an authorized operator) can invoke a withdrawal of their funds, and the contract will maintain an internal mapping of creator balances if it custodies tips. Unit tests will simulate various withdrawal scenarios to ensure the fee is correct and no more than the allowed amount can be withdrawn. In practice, if TipProxy already forwards funds directly to creators, WithdrawProxy might not hold user funds; however, it could still be used in an off-ramp scenario where a creator requests a payout to a bank: the WithdrawProxy could interact with the Circle API or a stablecoin bridge to facilitate off-ramping while deducting the platform fee.

DepositProxy (2.5% fee on fiat on-ramp): The DepositProxy contract handles incoming fiat-to-crypto conversions (on-ramps) and takes a 2.5% fee. When a fan purchases USDC via fiat (e.g. credit card through Circle or Revolut), the funds can be directed to this contract. The DepositProxy would then immediately take a 2.5% cut and credit the remaining 97.5% to the fan's on-chain wallet or custodial balance. For instance, if a fan on-ramps $100 worth of USDC, the contract would keep $2.50 worth of USDC as a fee and release $97.50 to the fan's address (or the platform's custodial wallet for that fan). This could be implemented by having the on-ramp service (Circle's payment flow) deposit into the DepositProxy contract address, with metadata indicating which fan or creator it's for. The contract, on receiving USDC (we can use an ERC20.transfer callback or a function invoked by the backend once funds arrive), will then transfer out the net amount. If the fan is tipping a creator directly via fiat, an alternative flow is used (see Circle integration below), but the DepositProxy is crucial for general top-ups. Like the other contracts, DepositProxy will allow an admin to configure the fee percentage and recipient address (for the fee). Unit tests will verify that various deposit amounts yield the correct fee deduction.

All contracts will be written in Solidity (>=0.8.x, which has built-in overflow checks). We will adhere to best practices for security: using OpenZeppelin libraries for math, ownership, pausing, and guarding against reentrancy. We will also include only the necessary functionality to minimize the attack surface. For example, since these are essentially fee-taking proxies, they won't implement complex logic beyond transferring tokens and emitting events. This simplicity helps security. The contracts will undergo unit testing (using a framework like Hardhat/ethers.js or Truffle) to simulate scenarios and ensure fees are computed correctly and funds flow to the right addresses. Additionally, tests will cover failure cases (like attempting to send a tip without prior token approval, or withdrawing more than available, etc.). Once tested, we will deploy the contracts to a testnet (likely Polygon Mumbai, given USDC usage on Polygon in the project) for integration. Deployment scripts will allow configuring the initial fee recipient (e.g. the TipJar+ treasury address) and verifying the contracts on a block explorer.

2. Circle API Integration

We will integrate multiple aspects of Circle's APIs to handle wallets, payments, gas sponsorship, and cross-chain transfers, ensuring a seamless fiat–crypto bridge and gas-less user experience. The integration includes:

Circle Wallets (DCW): Using Circle's Developer Controlled Wallets API, we create and manage custodial USDC wallets for users. When a creator registers, the backend will call Circle's API to create a new wallet for them (wallet type "GENERAL") and obtain a deposit address on the chosen blockchain (e.g. Polygon). The result (Circle wallet ID and USDC deposit address) is stored in our database for that user. This is done via an endpoint POST /circle/wallet/create (secured for authenticated users) – this endpoint was identified as missing and needed in the project documentation. We will implement it so that it checks if the user already has a wallet (to avoid duplicates) and then uses the Circle SDK/REST API to create one, providing an idempotency key for safety. On success, it saves the wallet ID and address to the user profile (and possibly marks the user's Circle setup as complete). A corresponding GET /circle/wallet endpoint will return the stored wallet details for the

logged-in user, so the frontend can display the deposit address (e.g. to allow the creator to receive direct on-chain tips to this address or to show QR codes). Both endpoints require JWT auth and proper role (e.g. only creators for now, though fans can also have wallets). If a user doesn't have a wallet yet, the GET will return a 404 or 409 error as specified.

Gas Station & Paymaster (Gas Fees in USDC): To improve usability, TipJar+ leverages Circle's solutions to handle blockchain gas fees. Circle offers a Gas Station service that can sponsor gas for transactions made via their wallets, and an ERC-4337 Paymaster contract to allow gas fees in USDC for smart contract interactions. We will integrate these so that neither creators nor fans need to hold MATIC/ETH for gas. For internal transfers and withdrawals done through Circle's APIs (e.g. moving USDC from a fan's Circle wallet to a creator's address), Circle's infrastructure can cover the gas and charge a fee for it. Specifically, Circle's Gas Station allows the developer to pay for transaction gas costs in fiat or via their account, with Circle typically charging around a 5% fee on the gas amount for this convenience. We will ensure our Circle API calls (such as createTransfer for internal tips or withdrawals) request gas sponsorship – in the Circle SDK, this is achieved by specifying a fee level (e.g. "medium") which lets Circle handle the gas. For example, when our backend calls Circle to transfer USDC from one wallet to another or to a blockchain address, we include fee: { type: 'level', config: { feeLevel: FeeLevel.Medium } } in the request, which means Circle will use their Gas Station to cover the network fee and later bill us (the fee is small relative to network costs).

In addition, for user-initiated transactions with our own smart contracts (TipProxy, etc.), we plan to integrate Circle Paymaster on supported networks. Circle Paymaster is an on-chain contract/system that allows users to pay gas fees in USDC instead of MATIC/ETH. This is particularly useful if we implement account abstraction or meta-transactions for a smooth UX. Circle's Paymaster charges a 10% fee on the gas cost for this service. While this is optional, we will prepare our contracts and front-end to support gasless interactions via Paymaster on networks like Arbitrum or Base (should we extend there). For instance, if a fan connects with a smart wallet (ERC-4337), the transaction to call TipProxy could be sponsored by the Paymaster, deducting the gas fee in USDC from the user's balance. In summary, Gas Station will cover gas for Circle wallet transactions (internal tips, payouts) at ~5% fee, and Circle Paymaster will enable covering gas for direct on-chain interactions at ~10% overhead – these costs will be factored into our fee model or absorbed by the platform as needed. By combining these, we fulfill the "pay gas in USDC" feature seamlessly, so users "don't need to hold native tokens for gas".

On-Ramp (Fiat → USDC with fee): For fans (or guest users) who want to tip using fiat (credit/debit card), we integrate Circle's Payments API. The flow uses Circle's Hosted Checkout feature for simplicity. We will implement an endpoint like POST /tips/onramp or reuse the guest tip endpoint for this. When a guest chooses to pay by card, the frontend will call our backend with the desired tip amount and creator info. The backend will verify the request (e.g. amount > 0, currency USD) and ensure the fan has a Circle wallet – if the user is truly a guest (not logged in), we will create a temporary fan account and wallet behind the scenes so that we have a destination for funds. (If the guest provided an email or if we force account creation for fiat payments, this wallet could belong to a newly registered fan user; otherwise, we might use a generic "guest wallet" owned by the platform to route the payment.) After that, the backend calls Circle's Payments API to create a Hosted Checkout

session for the specified amount and USDC as the output. This returns a checkout URL which we send back to the frontend. The frontend will redirect the user to this URL (opening Circle's checkout page in a new tab or modal) where the user enters their payment details. Circle will process the card payment and (upon success) convert the charge into USDC. According to the plan, once the payment is captured, Circle will send a webhook event (payment.capture.succeeded) to our backend. We will handle this webhook (see Webhooks below) to complete the tip flow: when we get the success event, our server knows which creator and amount it corresponds to (we can store a pending Tip record or use Circle's metadata on the payment to encode tip details). The backend then initiates the internal transfer of USDC to the creator's wallet. Specifically, the flow could be: Circle credits USDC to our master wallet or the fan's temporary wallet; we then call our CircleService to transfer the appropriate net amount to the creator's Circle wallet address. In doing so, we deduct the 2.5% DepositProxy fee. For example, if the fan paid $50, we receive ~50 USDC minus processing fees; from that, we take 2.5% (1.25 USDC) as our platform's on-ramp fee and transfer 48.75 USDC to the creator's wallet. (If the fan was fully a guest, we might treat the entire $50 as going to the creator minus fee; if the fan now has an account with a balance, we could also credit their balance and then do a normal tip, but the end effect is the same net to creator.) This on-ramp integration ensures even unregistered users can seamlessly support creators with fiat, while the platform still earns its fee and converts the funds to crypto. We will document and handle edge cases, such as payment failures or the user abandoning the checkout – those will be signaled via webhooks or not at all, and our system will cancel or expire pending tips accordingly.

Off-Ramp (USDC → Fiat payouts): For creators wanting to cash out their earnings to fiat (e.g. to a bank account), TipJar+ will use Circle's Payouts API. In the current implementation, creators can already withdraw crypto to their own wallet address (on-chain). We extend this by allowing withdrawal to fiat accounts. A creator could provide bank details (ACH or wire instructions) in their profile. When they request a payout, instead of (or in addition to) transferring USDC on-chain, our backend will call Circle's Payouts endpoints to initiate a transfer from the creator's Circle wallet to their bank. This might involve creating a payout request with an idempotency key, specifying the destination (which must be added/verified beforehand via Circle's APIs for payouts), and an amount. Circle will then handle converting USDC to USD and sending it to the creator's bank or card. We will apply the WithdrawProxy 3.5% fee in this flow by reducing the payout amount – for instance, if a creator requests a withdrawal of 1000 USDC worth to their bank, the platform will subtract 3.5% (35 USDC) as a fee and initiate a payout of the remaining $965 worth. The backend already creates a Payout record in the database with status PENDING. After initiating the payout via Circle, we update it to PROCESSING along with the Circle transaction ID. The actual completion can take some time (especially for fiat payouts). Therefore, we rely on Circle webhooks for payout status updates. When Circle confirms the transfer is complete (or failed), we'll get a webhook (e.g. transfer.success or transfer.failed) and update the Payout record status to COMPLETED or FAILED. This off-ramp integration means creators can receive real money to their bank with minimal manual intervention. (Note: Off-ramp to fiat may be a later feature – initially, creators might withdraw to their personal crypto wallet on-chain, which is already handled via Circle's wallet transfer as above. But the architecture supports plugging in full fiat payouts as needed.)

Circle CCTP (USDC Cross-Chain Transfers): If TipJar+ expands to multi-chain support (e.g. allowing tips on Ethereum, Arbitrum, etc., not just Polygon), we will integrate Circle's Cross-Chain Transfer Protocol. CCTP allows burning USDC on one chain and minting it on another, through Circle's infrastructure. We could use this to move funds between Ethereum and Polygon, for example, if a creator's wallet is on a different chain than a fan's funds. The integration would involve calling Circle's CCTP endpoints or smart contracts (Circle issues burn and attestation events which a relayer listens to). While not an immediate requirement, our system design accounts for it (the DEFAULT_BLOCKCHAIN config can be adjusted, and Circle's APIs can specify a target blockchain for each transfer). This ensures TipJar+ is future-proof: if we need to route a tip from an Ethereum-based fan to a Polygon-based creator, we can burn the tip amount on Ethereum and have Circle mint it directly into the creator's Polygon wallet (minus fees).

Webhooks & Status Updates: A crucial part of the Circle integration is handling asynchronous events via webhooks. We will set up a secure endpoint POST /circle/webhook on our backend to receive events from Circle (we will register this endpoint URL in the Circle admin dashboard along with a shared webhook secret). The backend will verify incoming webhook signatures using HMAC SHA-256 (Circle provides a signature header that we compare against our secret), ensuring the request truly came from Circle and wasn't tampered with. Upon receiving a webhook, the server will parse the eventType and act accordingly. Examples of events we will handle:

payment.capture.succeeded: Indicates a Hosted Checkout or card payment was successful. Our handler will find the pending tip/deposit associated (by an ID we set in the payment's metadata or by matching amounts and timestamps) and then proceed to initiate the internal transfer to the creator. If this event corresponds to a guest tip, we likely hadn't marked the tip completed yet (it was PENDING), so we now mark it as COMPLETED and create the actual Tip record with status. If the event indicates a failure or cancellation, we'll mark the tip as FAILED and possibly notify the user.

wallet.transfer.completed: Sent when a blockchain transfer initiated via Circle (internal tip or withdrawal) is confirmed on-chain. Our backend will look up the corresponding Tip or Payout by Circle transaction ID and update its status to COMPLETED (and record the tx hash if provided). The current implementation already sets tip status to COMPLETED immediately after initiating the transfer, but we will refine this: we might set it to PROCESSING after initiation and wait for the webhook to flip to COMPLETED, ensuring stronger consistency. This webhook is crucial for creator payouts: as noted, after calling initiateWithdrawal we set status PROCESSING, and when we get transfer.completed, we update to COMPLETED. If a transfer.failed event comes, we update the record to FAILED and likely credit back the creator's balance in Circle.

Other events: We will also handle transfer.chain_failed, payout.failed, or any relevant error events by marking records as failed and alerting admins. Additionally, if Circle sends a payment.refunded or chargeback events for card payments, we should handle those (perhaps revoking a tip if a chargeback occurs).

Security for the webhook endpoint is paramount: beyond signature verification, we will implement measures to prevent replay attacks. This includes checking the id of each incoming event against a log of processed webhook IDs – if we receive the same ID twice, we ignore the duplicate (Circle may retry webhooks, so our endpoint must be idempotent). We will log all webhook calls for audit. Furthermore, our webhook handler will be careful not to perform heavy logic inline; for instance, if a webhook triggers an internal transfer, we might enqueue a job or at least not block the response while doing external API calls, to avoid timing out Circle. In case our server was down and missed a webhook, we can fall back to polling Circle's API for transaction status as a safety net (the docs mention either webhook or polling should be used to confirm completions). By handling these webhooks, the system stays in sync with external processes and maintains correct status for tips and payouts.

In summary, the Circle integration covers the full cycle: creating wallets for users, facilitating fiat on-ramps and off-ramps with appropriate fees, transferring USDC on-chain behind the scenes, and leveraging Circle's services (Gas Station, Paymaster) to abstract away blockchain complexities like gas and cross-chain issues. We will use the official Circle SDKs where possible (as seen in the code, e.g. CircleDeveloperControlledWalletsClient for wallets) and adhere to Circle's best practices for security (API keys stored in .env, etc.). All secrets (API key, webhook secret, etc.) will be loaded from configuration and never hard-coded.

3. Backend (NestJS)

The backend is built with NestJS and will be extended to support all required endpoints and logic for the tip flow. We will organize the code into modules (e.g. TipsModule, PayoutsModule, CircleModule, UsersModule, etc.) following NestJS conventions. The main additions and changes include:

Tip Endpoints (Authenticated & Guest): We already have a TipsController with POST /tips for logged-in fans and POST /tips/guest for guests. We will finalize these implementations. For authenticated users, the createTip endpoint expects a JSON body with creatorId, amount, optional message and anonymity flag. It uses JWT auth (AuthGuard('jwt')) to get the fan's identity and then calls TipsService.processNewTip() with the fan's ID. This flow will remain mostly as is, but we will adjust the fee percentage to 3.5% (the code currently uses 2% for logged-in fans) and ensure the platform fee and net amounts are calculated accordingly. The processNewTip method in TipsService will be updated to use 3.5% (0.035) for any fan tip and 2.5%+ perhaps additional for guest (or treat guest differently as currently 10%). We will likely unify this by removing the 10% guest fee in favor of the new scheme: when fanId is null (guest), we'll handle the fee via the on-ramp process rather than a flat 10%. The service will verify the creator exists, is a Creator role, and has a Circle wallet ready. It will also validate the amount is a positive decimal with up to 6 decimal places (thanks to class-validator on the DTO). If the fan is logged in and has a Circle wallet, we call CircleService.initiateInternalTipTransfer(fanWalletId, creatorWalletId, netAmount) which executes the internal USDC transfer via Circle. This returns a Circle transaction ID and possibly a blockchain tx hash. We update the Tip record in DB to status COMPLETED (or PROCESSING if we choose to wait for webhook) with that info. All of this will be enclosed in a try/catch – if any step fails, we mark the tip FAILED and throw an appropriate exception.

For the guest tip endpoint (POST /tips/guest), the flow will be: verify the input (amount, creatorId, and a paymentGatewayToken or request for payment). We expect the frontend to call this only to initiate the Hosted Checkout. So in implementation, if a paymentGatewayToken is provided (for example, if we had a direct token from a payment widget), we could charge it directly – but since we use Hosted Checkout, instead we might interpret this endpoint as “create a payment session”. The service will ensure the creator is valid and then (as described in Circle integration) create the Circle checkout. We’ll likely move that logic to a new service function, e.g. CircleService.createCheckout(amount, creatorId, maybe fanId). That will call Circle’s Payments API (through an SDK or HTTP call) to create a checkout and return a URL. We then respond to the frontend with something like { checkoutUrl: "https://circle.com/checkout..."}. The frontend will handle redirecting the user. Meanwhile, we create a Tip record in the database with status PENDING and note that it’s a guest tip awaiting payment. We might also generate an internal reference (like a UUID) and include it in the Circle payment metadata so the webhook can identify which tip to finalize. This endpoint will not directly mark anything as completed – it’s an initiator. Security: this route doesn’t require auth (since guest), but we will implement measures like rate limiting or CAPTCHA if needed to prevent abuse (someone could otherwise spam payment links). Also, we only allow certain origins via CORS to call it.

Payout Endpoints: Creators will have an endpoint to request payouts of their balance. In the PayoutsController, we have @Post('payout') under the creator route, meaning the full path is POST /creator/payout. This is JWT-protected (only logged-in creators). We have a DTO with just an amount (string) and destinationAddress for the payout. The PayoutsService.createPayout(creatorId, amount, destination) is called to handle this. We will ensure this service method implements the 3.5% fee on withdrawal: when a creator requests amount, we might interpret that as the gross amount they wish to withdraw (so they will actually receive 96.5% of it after fee), or we could treat it as net amount and deduct fee on top. It’s more transparent to treat it as gross: e.g. if they request 100 USDC, they get 96.5 USDC to their address, 3.5 USDC goes to platform. Our PayoutsService will handle this by perhaps splitting the amount into amountNetForCreator and feeAmount similar to tips. However, since the code currently doesn’t do that (it just initiates withdrawal of the full amount), we might incorporate the fee by adjusting the amount before calling Circle. One approach: reduce the amount in the Circle transfer by 3.5% and separately transfer the 3.5% to the platform’s wallet (either via another Circle transfer or if the funds are on-chain, via smart contract). If both creator and platform have Circle wallets, an internal transfer of the fee portion to the platform’s wallet can be done just like a tip. We will implement this logic so that the fee is captured. After initiating the payout to the creator’s address (destinationAddress could be a crypto address or a bank token depending on context), we create a Payout record with status PENDING, then update it to PROCESSING once Circle accepts the transfer request. The response to the API call will be the Payout record (or at least confirmation). The actual completion relies on webhooks as described. If the platform supports direct fiat payout, destinationAddress might actually be something like a bank ID; in that case, Circle’s API and webhooks cover the rest.

On-Ramp/Deposit Endpoints: As part of the guest tipping flow, we will implement any missing endpoints for initiating deposits. If using Hosted Checkout, the POST /tips/guest doubles as the deposit initiation. However, we might also expose a more general endpoint

for logged-in fans to top-up their wallet. For instance, POST /fan/deposit with an amount could create a Circle checkout for the fan to add funds to their own wallet. This would be similar to the guest flow but ties to the fan's identity. The strategy document mentions allowing a fan to *"doładować USDC na portfel twórcy za pomocą karty (fiat→USDC)"* in one go (guest tipping directly). It also hints that after creating the payment, we return a URL and the frontend should redirect, and that if the fan had no wallet, we create it on the fly. We have covered these in the guest flow. We'll ensure any such endpoint is implemented as needed and documented.

Circle Webhook Endpoint: We will add a new controller (e.g. CircleController in CircleModule) with a route POST /circle/webhook to handle incoming webhooks. This will be configured as an unprotected route (no auth, since Circle's servers won't have our JWT), but we will secure it by verifying the signature as described. The handler will parse the JSON payload and call a service method (perhaps CircleService.handleWebhook(payload)). That service will implement logic for each relevant event type. For example, on payment.capture.succeeded, find the pending tip record and complete the tip: update its status to COMPLETED, set processedAt timestamp, and call CircleService.initiateInternalTipTransfer to the creator's wallet for the net amount (if not already done). On transfer.completed, update the corresponding Tip or Payout status to COMPLETED. We will also log unexpected event types and possibly ignore those we don't need. The webhook handler will respond with a 200 OK quickly after kicking off necessary processing, so Circle knows we received it. We'll also implement idempotency: if the same eventId comes again, we won't double-process (possibly by keeping a cache or DB record of processed webhook IDs). All secrets for validating (Circle provides an HMAC header with each webhook) are stored in our config and used here. The documentation emphasizes validating that the webhook truly comes from Circle, which we will do by computing the HMAC of the payload with our shared secret.

Database (Prisma) updates: We will extend our Prisma schema if needed. The current schema has models for User, Tip, Payout, etc., with status fields. We might add fields to Tip for paymentId or metadata to track external payment references, and to Payout for destination details (like bank info or crypto address). We will also ensure that when a tip is completed or a payout is completed, we record the processedAt time and any transaction hash or Circle IDs. The Tip model already has circleTransferId and blockchainTransactionHash fields for this. We'll use those to store results from Circle.

Authentication & Roles: The NestJS app already uses JWT for auth (with cookies) and defines roles (UserRole: FAN, CREATOR, ADMIN). We'll ensure that: only creators can hit certain endpoints (like /creator/payout or maybe /creator/profile update), only fans/creators themselves can access their own data, etc. Endpoints like /tips/guest will be open but with careful validation. We will also use Nest's built-in features to prevent CSRF since we rely on cookies for JWT – for instance, the JWT strategy is extracting from cookies, so for state-changing requests from a browser, we will implement a CSRF token pattern (the server can issue a CSRF token in a cookie or header and require it in requests). This wasn't initially in scope but is mentioned under security, and it's important because with cookie-based auth an attacker could try CSRF; by requiring a custom header or token (Double Submit Cookie), we mitigate that.

Logging and Error Handling: We will use NestJS's Logger to log important actions (tip created, payout initiated, webhook received, etc.) including IDs for traceability. Errors will be converted to proper HTTP exceptions with user-friendly messages (the existing code already throws 404 with Polish messages like "Portfel fana nie jest skonfigurowany" which we will keep). We'll ensure that any external API errors (Circle API failures) are caught and result in either a 502 Bad Gateway or 500 with a clear message, rather than crashing. For example, if Circle returns an error during wallet creation or transfer, our service catches it and calls a helper to translate it to an HttpException.

By implementing these backend features, the server will be capable of orchestrating the entire tip flow: from onboarding (wallet creation), to tipping (internal or via fiat), to withdrawals, all while maintaining a record in the database and enforcing the business logic (fees, limits, security checks). We will maintain the code structure in English (variables, function names, etc. remain in English as in the codebase), but will provide Polish-language responses or messages where appropriate (the code already has Polish messages for exceptions). All new endpoints will be documented in the README (in Polish) for clarity.

4. Frontend (Next.js)

On the frontend, we will create a Next.js application (if not already created) or add the required pages and components to an existing one. The frontend is responsible for providing a smooth UI/UX for creators setting up their profiles and fans sending tips, aligning with the design guidelines provided (as per the UI/UX documentation). We will structure the frontend code into pages and reusable components, likely under directories like pages/creator, pages/fan, and components for shared UI elements. Key front-end elements:

Creator Setup Page (/creator/setup): After a creator signs up or logs in for the first time, they will be directed to the setup page. Here, they can configure their profile and link their payment details. The form will allow uploading a profile avatar and banner, entering a display name and bio, and possibly setting a fundraising goal (e.g. "collecting 1000 USDC for a new camera"). On submit, the frontend will call the backend endpoints to update the user profile (e.g. PUT /creator/profile) and crucially to create their Circle wallet. We will automatically call POST /circle/wallet/create when the creator finishes setup – this returns the new wallet's ID and USDC address. The UI can display a confirmation (maybe showing the last few characters of their deposit address for transparency). We'll ensure to handle error cases: if wallet creation fails (e.g. Circle API issue), show an error and possibly retry or instruct the user. Once setup is complete, the creator is ready to receive tips.

Fan Setup (/fan/setup): If we allow fans to have accounts (for example, to maintain a balance or track their tips), a similar setup page can be provided for fans. This might simply generate their Circle wallet in the background (if not done at registration). Fans might not need to input much info aside from maybe choosing a username or linking social accounts for login. If the platform supports Web3 login (Sign-In with Ethereum via MetaMask), a fan might connect their MetaMask which gives them a non-custodial wallet; in that case, we wouldn't create a Circle wallet unless they want to top-up via fiat. We'll handle those conditions: the UI can present an option "Create TipJar Wallet for easy tipping" for crypto-connected users. The fan setup is mostly about enabling easier tipping (so likely it

ensures a Circle custodial wallet exists for them if they want to use the internal balance method).

Public Profile Page (/[username]): This is the main page where fans (logged in or guests) can go to tip a creator. It is accessible at a URL with the creator's username. We will implement this as a Next.js dynamic route [username].tsx that fetches the creator's profile data from the backend. The page will display the creator's branding and tip interface as designed. Visual layout: at the top, we show the creator's banner image, their avatar, username, and bio, establishing their identity and building trust. Below that, if the creator has set a fundraising goal, we display the goal and a progress bar indicating how close it is to completion (we'll get the current total of tips from the backend or calculate from tip history). A section for "support history" may show recent tips: e.g. "JaneDoe tipped 5 USDC – Great content!" for social proof. This can be implemented by fetching recent tip records (the backend could provide /creator/{id}/recent-tips).

Next on the page is the Tip Widget – a central UI component allowing the user to select an amount and a payment method. We will implement the slider input that lets the fan pick a custom amount intuitively. The slider might be configured from 1 USDC up to, say, 100 USDC (we can adjust range based on some logic or the creator's settings). As the user drags the slider, we show the selected amount and maybe a dynamic color fill on the slider track for feedback. Additionally, we provide quick preset buttons (like $1, $5, $10) so the fan can click those for convenience (to avoid "analysis paralysis" from an open-ended slider). The combination of slider and preset buttons addresses different user preferences, as noted in the UX analysis, minimizing friction in decision-making. The tip widget also includes a text field for an optional message and a toggle for anonymity (so the fan can choose to show their name or tip as "Anonymous").

Below the amount selection, the user sees payment options. If the user is logged in and has a TipJar wallet (Circle wallet), one option will be "Tip from my TipJar Wallet" – essentially using their custodial balance. We will show their current balance on that button (fetched via GET /circle/balance which uses CircleService to get the wallet's USDC balance). If their balance is insufficient for the chosen amount, we might disable that option or prompt them to top-up. Another option will be "Credit/Debit Card" (potentially with icons for Visa/Mastercard) and possibly "Google Pay / Apple Pay" if Circle's Hosted Checkout or our integration supports those (the UI mock suggests options for card and Google Pay)

. Guest users will obviously only have the card/Google Pay options. Logged-in fans could use those too if they prefer not to use internal balance. There might also be an option "Crypto Wallet" if a fan connected via MetaMask and wants to pay directly in USDC from their wallet – in that case, the flow would involve calling our TipProxy contract. We will include a button for "On-Chain Payment" or "Pay with MetaMask" which triggers a web3 transaction (this would call TipProxy's sendTip). This gives maximum flexibility: users can tip via TipJar wallet, via fiat, or via their own crypto.

When the fan clicks one of the payment buttons, we handle accordingly:

TipJar Wallet (Internal transfer): We call our backend POST /tips endpoint with the amount, message, etc. If that returns success, we display a success state (maybe a confetti

animation or just a thank-you message) and update the page (e.g. add the tip to the recent supporters list). Since that endpoint results in an immediate transfer and completion in most cases, we can optimistically update the UI. We will also subtract the tipped amount from the fan's displayed balance.

Card/Google Pay (Fiat): We call POST /tips/guest (or /tips/onramp) with the amount and creator. We receive the checkoutUrl from our backend and then open it. We'll likely show a modal overlay with a message like "Redirecting to payment…" and open the Circle Hosted Checkout in a new tab or an embedded iframe. Once the user completes payment, Circle will redirect them to a success page we configure (possibly a TipJar domain page that simply says "Payment received, you can close this window"). Meanwhile, our backend will process the webhook and finalize the tip. To give feedback to the user on the original page, we can do one of two things: use webhooks to update UI (hard in real-time on a static page), or simpler, poll our backend for the tip status. For example, after redirecting the user to checkout, our page can periodically call GET /tips/status?tempId=XYZ to see if the tip went through. Or we show a message like "Your tip is being processed and will appear soon." Since the Hosted Checkout flow is somewhat out-of-band, we might not get immediate confirmation on the front-end. However, we can improve UX by asking the user to come back after payment; when they return, the page will show the new tip in history. We'll ensure that this flow is communicated clearly to avoid confusion.

Crypto Wallet (MetaMask): We will use a web3 library (like Ethers.js or web3.js) in the frontend to request a transaction. The user will hit "Pay with MetaMask", then their MetaMask will prompt to approve the TipProxy contract to spend the amount (if not approved already) and then another transaction to call sendTip(creatorAddress, amount). We will have to get the creator's payout address – if the creator uses custodial wallet, that address is the Circle deposit address (which we have from backend). We will supply that to the contract call. The contract will then handle the fee and forward funds. After the transaction is mined, we show a success message. (Note: If using account abstraction, this flow could instead craft a UserOperation and send to bundler with Paymaster covering gas; but on frontend it will appear similar except no gas asset needed from user).

The public profile page will be implemented with either Server-Side Rendering (SSR) or Static Generation (SSG) for the public data, as suggested by the analysis. We can use Next.js getServerSideProps to fetch the creator's profile (name, bio, images, goal, etc.) from our backend (GET /creator/profile?username=). SSR ensures the page loads quickly and is SEO-friendly (important for discoverability of creators). The tip widget portion that is interactive will be client-side hydrated React. For dynamic data like current balance (for logged in fans) or recent tips, we might use SWR or WebSockets. A simple approach: fetch recent tips via an API route when the page loads and periodically refresh it if the user stays on the page, so new tips appear in the support history in near real-time.

QR Code Generator: We will implement a page or component that allows a creator to get a QR code for their profile. This might be under the creator's dashboard (e.g. /creator/qr). It will simply take the URL of their public profile (tipjar.plus/@username) and generate a QR code image that encodes that URL. We can use a library like qrcode.react or an API to generate the image. The UI will show the QR code and perhaps a download button so the creator can

save it and share it (on streams, print on materials, etc.). This QR code, when scanned, leads a fan directly to the creator's public profile where they can easily tip. Because our public profiles are web-based, anyone scanning the QR can use the tipping interface – no app install needed, which aligns with the goal of low friction tips.

UI Components: We will create reusable React components for various parts of the UI:

Avatar and Banner: Component to display a user's avatar (circular image) and maybe status (if we show if they're online for live streams, etc.). Also a component for the banner image with proper aspect ratio. We'll ensure images are sanitized (use Next Image for optimized loading) and provide fallback images if not set.

Bio and Social Links: A component to display the bio text safely (we'll strip any disallowed HTML to prevent XSS, although we likely only allow plain text bio). If the creator provided social media links, we display icons (Twitter, YouTube, etc.) linking to those.

Tip Amount Slider: A specialized component encapsulating the slider and amount presets. It will manage the state of selected amount and provide a callback when changed. We'll also style it according to the design – possibly using Tailwind CSS or styled-components for the gradient fill effect as described.

Payment Method Buttons: Component that shows the available payment methods (Wallet, Card, Crypto, etc.) in a responsive way. It will take props like onChooseMethod(method) to notify the parent which was selected. It might also display the fan's balance on the Wallet button (prop for balance).

Tip Modal/Confirmation: We might use a modal to confirm with the user if needed (e.g. "Are you sure to tip 50 USDC?") especially for large amounts, or to show processing state. But since simplicity is key, we may skip a confirm for small amounts to reduce clicks.

Payout Component: On the creator's dashboard, a section showing their current earnings and allowing withdrawal. This will fetch their balance (the Circle wallet balance via our backend, plus perhaps any off-chain pending amount). It will have a form to input an amount to withdraw and a destination (if multiple options like on-chain vs bank). On submit, call POST /creator/payout. Then show status (initially "Processing" until webhook confirms). We'll also list past payouts with statuses (from DB via GET /creator/payouts). This helps creators track their money.

Balance Display: For both creators and fans (if fans have persistent wallets), a simple component to show "Your Balance: X USDC". This will be updated after any tip or deposit.

Security/UI: We'll ensure all form inputs and outputs are handled safely. For example, use React state for forms, and escape any dynamic text we render (Next does this by default in most cases). We'll use Next.js built-in protection against XSS by not dangerously setting HTML except where necessary (which is nowhere in our use-case aside from maybe rendering bio with line breaks). We will also ensure to use HTTPS for all API calls and serve the app over HTTPS (especially since we handle payments). CORS will be configured so our frontend domain can reach the backend API.

Styling and Readability: We will follow the provided design (which seems clean and modern). Possibly we'll use a CSS framework like Tailwind (if already in use) or styled JSX/CSS modules in Next. The UI/UX guide emphasizes a streamlined user experience that reduces cognitive load and builds trust. We will incorporate that by clear messaging, e.g., showing the creator's successes ("X supporters so far!") and making the call-to-action (the tip button) very prominent. The text on the site will be in Polish for user-facing strings (as the platform is presumably Polish-focused), e.g. "Wesprzyj" for support, "Wyślij napiwek" for send tip, etc., matching the tone of the existing messages.

Overall, the frontend will be packaged in the frontend/ directory with a Next.js project structure. The pages we implement – /creator/setup, /fan/setup, /[username] – correspond to the key flows. We'll also have a /creator/dashboard page for creators to see their stats (tips received, balance, payouts) – this can reuse many components. Navigation will be simple: creators after login go to their dashboard/setup, fans go to maybe a home or directly to creators' pages via links. We will add a navigation bar with login/logout and perhaps language toggle if needed.

By building these front-end pieces, we ensure an intuitive and engaging experience: a potential fan can scan a QR code or click a link, see a polished profile that instills confidence (with personalized branding and social proof), easily select an amount via a fun interactive widget, and complete payment in a method of their choice – all in a matter of seconds. This directly supports the platform's goals of removing friction and encouraging even small contributions.

5. Security Considerations

Security is critical across all parts of TipJar+. We address it in several layers: smart contracts, backend, and frontend:

Smart Contract Security: The contracts will undergo careful review and include standard protections. We use OpenZeppelin's libraries for ownership and pausable functionality, enabling the platform to pause tipping or withdrawals in case of an emergency (e.g. a vulnerability or suspected attack). Functions that transfer funds (TipProxy's tip function, WithdrawProxy's withdraw) will be marked nonReentrant to prevent reentrancy attacks on external token transfers. We will also restrict function access appropriately: for example, configuration functions (setting fee percentages or addresses) will be onlyOwner. The fee calculations use integer math on smallest units (wei or in USDC's case, 6 decimals) to avoid precision errors. In testing, we'll include scenarios to ensure no integer overflow or underflow (Solidity 0.8 auto-checks, but we remain vigilant). Additionally, when interacting with ERC-20 tokens, we will use safe methods (OpenZeppelin's SafeERC20 library) to handle non-standard tokens. While we will primarily accept USDC (a well-known stablecoin contract) in these contracts, we might allow an upgrade to support other stablecoins, so our code will not assume USDC's address is hardcoded – it will be configurable, and only that token can be used for tips (enforced by checking msg.sender or token address in transfer

callbacks). By limiting to one known stablecoin, we reduce risk of unexpected token behavior.

Backend Security: The NestJS backend will include input validation and robust authentication checks. All endpoints that modify state or return sensitive info will require a valid JWT. We use class-validator on DTOs to ensure inputs meet expected formats (e.g. valid UUIDs, decimals, strings of certain length). We will implement rate limiting on the important endpoints: for instance, the POST /tips/guest creating payment sessions should be limited per IP to prevent abuse, and the webhook endpoint will only accept traffic from Circle (we can check the source IP against known ranges in addition to HMAC verification for extra safety). We will enforce CORS such that only our frontend domain can call the APIs from a browser, preventing other sites from potentially invoking our endpoints with the user's credentials. Moreover, since our JWT is stored in httpOnly cookies (for web), we will implement CSRF protection. One way is to use NestJS's built-in CSRF protection or a custom solution where we expect a custom header (X-CSRF-Token) with a value that must match a cookie value. This stops malicious third-party sites from tricking a user's browser into executing actions on TipJar+.

The backend will also sanitize any data that goes into responses. For example, if a creator's username or bio is used to form dynamic content (like an HTML page title or meta tags in SSR), we will escape it to prevent HTML injection. Our use of Prisma ORM helps prevent SQL injection, as queries are parameterized. We'll also ensure that any user-generated content (like tip messages) is filtered for offensive or unsafe content if needed (could be a future enhancement).

Financial limits: We will implement server-side checks for amount limits to mitigate abuse. For example, we might set a minimum tip amount (like 0.01 USDC) and a maximum per tip or per day to avoid erroneous large transactions or potential money laundering flags. These limits can be configured and adjusted. If a tip amount is outside allowed range, the backend will reject it with an error. Similarly for payouts, we can impose a cap per day. These are business rules that also have security implications (large unexpected transactions could be fraudulent).

Replay Attack Prevention: The Circle webhook signature verification and tracking of event IDs guards against replay of external events. For internal actions, since we use idempotency keys when calling Circle (e.g., we generate a random UUID for each transfer or payout), Circle will not execute the same transfer twice even if our request is accidentally resent – this is another safety net. Our backend will also use idempotency on our side for certain actions (for instance, if a user double-clicks the tip button causing two requests, we might lock the tip process by user or use a unique token to ensure only one goes through).

We'll store secrets (DB password, JWT secret, Circle API keys) in environment variables and never commit them to the repo. The .env file provided will be used, but in deployment we use secure env storage. The JWT secrets will be sufficiently long and random to prevent brute force. Passwords (if any traditional login, although we mostly use OAuth or web3) will be hashed with a strong algorithm (bcrypt).

Frontend Security: On the client side, we avoid introducing XSS vulnerabilities by not injecting raw HTML. All data from the backend is treated as untrusted. We'll use React's default escaping or libraries to sanitize where needed. For example, when rendering the creator's bio, if we allow some rich text (maybe not in MVP), we'd sanitize it. Currently, likely bio is plain text, so a simple {bio} is safe. We will also make sure not to expose sensitive data in the client (e.g., the Circle API keys never touch the frontend; all Circle interactions go through our backend).

The frontend will use HTTPS for all requests to avoid MITM eavesdropping. We'll also ensure that if the user is on an unsecured connection we upgrade or warn (HSTS can be enabled on hosting). We'll audit our use of any third-party scripts; e.g. if we embed any external widget, we'll ensure it's from a trusted source.

CSP (Content Security Policy): We may add a CSP header to our site to restrict sources of scripts, frames (we will allow Circle's domain if using Hosted Checkout in an iframe or redirect). This can mitigate XSS by disallowing inline scripts or unknown domains.

In addition, we consider privacy: if a fan tips anonymously, we will honor that by not revealing their identity in the public feed. And we protect user data: personal info (like emails, if any collected) won't be exposed via API responses to other users.

By addressing security at all these points – smart contracts (preventing theft of funds), backend (preventing unauthorized actions and ensuring integrity of transactions), and frontend (preventing injection and protecting user sessions) – we aim to make TipJar+ a safe platform for financial transactions. The strategy includes defense in depth: even if one layer (say the frontend) is compromised, the backend checks and smart contract limitations still safeguard the core assets. We will document any security-related configurations (like setting up webhook secrets and JWT secrets) in the README for the deployer to follow. Finally, before going live, we will conduct testing specifically for security: attempt malicious inputs, check that webhooks can't be forged, etc., to validate these protections.

6. Project Structure & Output

The implementation will be organized into the following folder structure for clarity and maintainability, aligning with the requested division:

contracts/ – Contains the Solidity smart contracts and related files. We will have TipProxy.sol, WithdrawProxy.sol, and DepositProxy.sol here, possibly along with an ERC20FeeProxy.sol base if we factor out common code (for example, TipProxy and WithdrawProxy share similar fee logic). A subfolder contracts/test/ (or a separate top-level test/) will include unit test scripts for the contracts. We'll use Hardhat, so expect a hardhat.config.js and test files in JavaScript/TypeScript that deploy the contracts to a local EVM and run scenarios (these tests ensure fees are correctly taken and edge cases handled). We will also include a deployment script (in contracts/scripts/ maybe) for testnet deployment, which can be used to deploy all three contracts and output their addresses. Configuration like the platform fee recipient address or initial fees can be set via constructor

or an initialization function – our script will set those (likely using the owner's address and the default fees 3.5% or 2.5%).

backend/ – The NestJS backend source code. Inside, we follow Nest's convention with a src/ directory. The modules will be structured as:

src/tips/ – containing tips.controller.ts, tips.service.ts, create-tip.dto.ts (for both CreateTipDto and CreateGuestTipDto), and unit tests tips.service.spec.ts etc. This module handles tip creation logic. We will update these files according to our new logic (fees, calling Circle, etc.) and ensure all tests pass (we'll write new tests especially for the guest tipping flow with the hosted checkout – possibly mocking CircleService).

src/payouts/ – with payouts.controller.ts, payouts.service.ts, create-payout.dto.ts, and tests. This module manages creator payouts. We will update the service to handle fee deduction and perhaps add tests for fee calculation on payouts.

src/circle/ – with circle.service.ts, circle.module.ts, and we will add circle.controller.ts for the webhook and possibly wallet endpoints. The CircleService already contains methods for wallet creation, internal transfers, and withdrawals. We will extend it with methods like createCheckout and verifyWebhookSignature. Also, if needed, we might use Circle's official SDK (which seems to be included as @circle-fin/developer-controlled-wallets) for some calls, and direct REST calls for others (like Hosted Checkout, which might not be in that SDK). The controller will map routes: e.g. POST /circle/wallet/create -> circleService.createWalletForUser(), GET /circle/wallet -> returns DB info, POST /circle/webhook -> circleService.handleWebhook().

src/users/ – for user profile management. We might add fields to the User entity (like bio, avatar URL, etc.) and create endpoints to update those. For example, PUT /creator/profile in a UsersController. Also, a GET /creator/profile/:username to retrieve public profile info (which our Next.js SSR can call). These endpoints ensure the frontend can get the necessary profile and goal data for the public page.

Other existing modules: src/auth/ for authentication (we likely keep as is, using JWT cookies as configured), src/prisma/ for DB connection, src/shared/ for any utilities (e.g. maybe a common function to compute fees to avoid duplication). We also have src/overlay/ from the codebase which relates to a streaming overlay feature (displaying tip alerts on live streams). We will ensure that our new tip events tie into that if needed (for instance, when a tip is completed, the backend could emit via WebSocket or push to Redis pub/sub so that the overlay service can show "New tip from X!" on the creator's stream). This might already be partially implemented, but we can extend it so that tips coming from all flows trigger the overlay.

The backend will continue using Prisma for the database; we'll update the Prisma schema file (schema.prisma) if needed to add fields for user profile and any new models (perhaps a Transaction log model or extension of Tip to include more info). After changes, we run prisma migrate to update the DB (instructions in README).

frontend/ – The Next.js application. Inside frontend/pages/, we'll have creator/setup.tsx, fan/setup.tsx, [username].tsx (for public profiles), possibly creator/dashboard.tsx, and so on. Inside frontend/components/, all the UI components (Avatar, TipWidget, etc.) will reside. We might also have frontend/pages/index.tsx as a landing page (could be simple marketing or redirect to a login). Next.js configuration and dependencies (package.json) will be set up to support our needs (we may add axios or use Next's built-in fetch for API calls). For state management, lightweight approach using React hooks and SWR (stale-while-revalidate) for fetching could be enough. We'll also implement any needed context providers, e.g. an AuthContext to hold the logged-in user info, so we know if a fan is logged in and their role. Next's API routes might not be used since we have a separate backend; all calls from frontend go to the NestJS API server. We will ensure the base URL and any required credentials (cookies) are properly configured (perhaps using withCredentials in axios).

creator/ and fan/ folders: The requirement lists these separately. It may refer to segregating parts of the code or deliverables specific to creators vs fans. Likely, in documentation or output packaging, we'll separate things like Creator-facing features vs Fan-facing features. For the code, much is shared, but we can logically separate UI and maybe some backend logic by role. For instance, under frontend/pages/creator/ vs frontend/pages/fan/. In the backend, we don't literally have separate folders for creator and fan (they use the same modules with role checks). However, to align with the request, we might prepare two separate documentation sections or environment configuration for creator vs fan. It's also possible they expect any scripts or utilities for creators (like a CLI or admin tool) under a creator dir. Given the context, we interpret it as mostly a way to organize deliverables, so we will ensure to clearly label which parts of the codebase cater to creators and which to fans.

circle/ – This could contain scripts or JSON configs related to Circle integration (for example, a collection of API request samples, or a dedicated README for how to set up Circle webhooks and keys). But since we integrated Circle in the backend code itself, we might not need a separate folder, unless to place some mock data or the developer's Postman collection. However, to be safe, we will include a circle/ directory containing documentation of the Circle integration: e.g. circle/webhooks.md describing how we handle them, or circle/api_samples/ with example payloads. If we wrote any custom scripts (for example, a script to backfill existing users with wallets, or to reconcile Circle balances) we could put that here.

utils/ – A folder for utility scripts or common libraries. For instance, we might have utils/fees.ts containing helper functions to calculate fees consistently. Or a utils/crypto.ts for HMAC verification of webhooks. On the frontend, utils could hold formatting functions (e.g. to format dates or amounts). We will populate this as needed to avoid code duplication.

Configuration & Environment: We will supply example environment files (like .env.example) listing all required env vars (like DATABASE_URL, CIRCLE_API_KEY, CIRCLE_WEBHOOK_SECRET, etc.). The README will explain how to obtain and set these. The backend's config will allow switching between a testnet (Polygon Mumbai) and mainnet (Polygon mainnet or others) by changing env variables (like DEFAULT_BLOCKCHAIN which is set to 'MATIC-AMOY' by default for test environment – we'll clarify how to set it to mainnet when deploying live).

README & Documentation: We will provide a comprehensive README (in Polish) covering how to set up and run the system, as well as a user guide for the features. The README will be placed at the root of the output, and possibly broken into sections or separate files for clarity (e.g. README.md for overview and instructions, plus docs/Architecture.md for deeper explanations). It will include:

Project overview and goals (summarizing TipJar+ tipflow functionality).

Instructions to install dependencies and run the backend (NestJS) and frontend (Next.js) locally. For example: how to run npm install (with reference to the provided package-lock.json), how to set up the database (running migrations, possibly seeding some data for testing), and how to start the dev servers.

Smart contract deployment instructions: e.g. how to run Hardhat tests (npx hardhat test) and how to deploy to a testnet (npx hardhat run scripts/deploy.js --network mumbai). We will include the addresses of the deployed contracts on the testnet and how to configure the backend with those (the backend needs to know the TipProxy address if it ever interacts with it, though in our design the frontend calls TipProxy directly; however, the backend might need the WithdrawProxy address if it triggers withdrawals via contract, etc.).

How to configure Circle API: obtaining API keys, setting the CIRCLE_API_KEY and CIRCLE_ENTITY_SECRET (if needed for wallets), setting up the webhook URL in the Circle dashboard to point to our /circle/webhook endpoint, and placing the webhook secret in our .env. We'll also mention any Circle account configuration like adding a default wallet set ID (Circle uses wallet sets to group wallets; the documentation suggests using the proper CIRCLE_WALLET_SET_ID in requests).

Usage guide: how a creator registers (e.g. via Google OAuth or other – the code hints at Google sign-in support), how they then set up their profile, and how fans can find and tip them. This can be a step-by-step walkthrough from a user perspective, to ensure the testing team or stakeholders can easily try out the flows.

Explanation of the fee structure in simple terms (3.5% on tips and withdrawals, 2.5% on fiat deposits), so it's clear in documentation why the numbers in transactions appear as they do.

Any technical notes: e.g. "We rely on Polygon Mumbai for test environment – ensure you have Mumbai USDC contract address configured, etc.", or "The overlay feature requires a separate front-end (not covered here) to display alerts."

Finally, we will package the output in stages as requested. Each stage (every few hours) would be a zip containing the updated folders. The final deliverable will have all the folders (creator, fan, contracts, backend, frontend, circle, utils) with the implemented code and a Polish README ready to be placed into the repository. The code will remain in English (for consistency with existing codebase), while documentation and inline comments (where appropriate) can be in Polish to aid local developers. This way, the project TipJar+ will have a complete tipflow implementation that is ready for testing and deployment, combining

on-chain smart contracts with Circle’s Web3 services to deliver a frictionless tipping experience. All features described have been implemented according to the provided context and requirements, and the system is now prepared to be run end-to-end in a testnet environment for final verification.

Źródła: The implementation details and justifications above are based on the TipJar+ project files and strategy documents provided, including the API specification summary, the comprehensive UI/UX and architecture analysis, and external references for Circle’s services (e.g. Circle Paymaster announcement). All integrated knowledge has been tailored to the project’s needs.